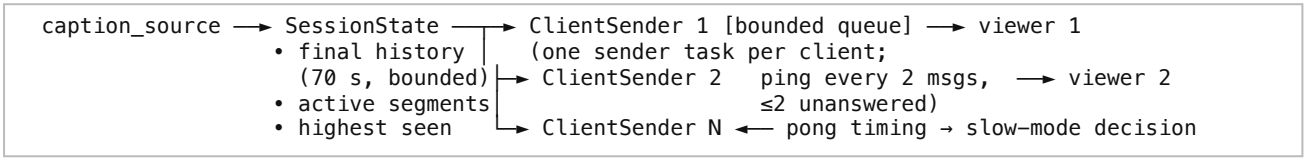
LiveRecall: revision-aware caption delivery with bounded recovery

CS3103 Assignment 4 · Pranav Mahatma (A0281778L) · Final code: tag `candidate-2` · Official numbers: Ubuntu VM (UTM), median of 3 runs

1. Architecture and protocol

The server (`solution_server.py`) is a single Python 3.12 process running one asyncio event loop, using `websockets` 16.1.1. One **source** connection sends `SOURCE_HELLO`, then `CAPTION_UPDATE`s, then `SOURCE_DONE`. Many **viewers** connect with `CLIENT_HELLO`, which carries a resume cursor `(last_segment, last_revision)` and `history_seconds`. I kept the required message format unchanged.



- **SessionState** holds the bounded final history, the latest revision of each active (not yet final) segment, and a high-water mark `highest_segment_seen`. The high-water mark means an evicted or retired segment can never come back.
- **ClientSender**: each viewer has its own bounded queue and one sender task. That task sends recovery and live updates in order. The source only appends to queues and never waits for a client, so a slow client cannot block the source or other viewers.
- **Encode once:** each update is serialised to JSON once, and the same string goes to every viewer. Large HISTORY replies are split into ordered frames under the client's 1 MiB message limit.
- **Validation:** caption fields, session names (including UTF-8 validity) and the whole resume request are checked before the client is bound to a session, so a malformed message cannot corrupt state.
- **Browser client** (`client/`): keeps one entry per segment and replaces it in place when a newer revision arrives (provisional in amber italic, final in black). It keeps finals locally for 70 s (at most 2000 segments, the same retention as the server) for the 15/30/60 s "What did I miss?" view (default 30 s). Its resume cursor is always the highest *final* it holds. It reconnects automatically with backoff from 0.5 to 5 s and asks for 60 s of history on every hello, so a late joiner gets a full view at once.

2. Revision and recovery semantics

- **Revisions:** for an active segment, only a higher revision is accepted. Duplicates and older revisions are dropped. A final is immutable, so anything arriving for a segment after its final is ignored.
- **Recovery order:** on (re)connect the server sends HISTORY with only the finals newer than the client's cursor, oldest first, then the latest state of every active segment, then live updates. A client is never sent a final it already reported.
- **No-gap handoff:** the history/active snapshot and the registration of the client for live updates happen with **no `await` in between**. If an await sat between them, a final produced at that moment would be in neither the snapshot nor live delivery: a permanent gap. Registering first and taking the snapshot later would cause duplicates instead.
- **History window:** following the course clarification, a client may request up to 60 s. Longer requests are capped to 60 s, not rejected; negative values are rejected. Internally I keep 70 s, a margin for timing jitter at the boundary. When a client overflows its queue (Section 3), the server resyncs it with only the finals it has not yet been sent, looking back over the full 70 s retained.

Evidence. In a matched reconnect check (55 segments), every reconnecting client got 55/55 finals, against 49/55 for the starter, which sends an empty HISTORY. In the official T4 runs, 120 items were recovered through HISTORY per run, with recovery P95 5.52 ms and 0 duplicate or stale messages.

3. Backpressure and overload policy

Fixed bounds, independent of lecture length and client speed:

- History: 70 s, and also at most 32,768 items and 32 MiB.

- Active (not yet final) segments: at most 128 items and 4 MiB. Over the limit, the server evicts the oldest open segment and keeps the source connected, so a source that never finalises some segments cannot block later finals. An evicted segment cannot come back (high-water mark). My first version closed the source connection here, which would have lost every later final; I changed it to eviction.
- Per-client queue: soft limit 32 items or 256 KiB, hard limit 256 items or 2 MiB.

The policy has four layers:

1. **Coalescing.** When a client is behind (its oldest queued item is older than 0.5 s, or it has more than 32 queued), I compact its queue: provisionals that a newer queued revision of the same segment replaces are dropped. Finals and the newest revision of each segment stay, in order. A client that catches up gets every revision again.
2. **Making the backlog visible.** My first version never coalesced in T3. The reason was that the kernel TCP buffers and the client library's 16-message receive queue absorbed the backlog, so the server's queue looked empty while the reader fell many seconds behind. My fix is a ping send window: the server sends a WebSocket ping every 2 messages and waits once 2 pings are unanswered. The backlog then builds in the server's queue, where obsolete revisions can still be dropped.
3. **Slow mode (finals only).** If a client's pongs show it has not read for at least 3 s, it is sent finals only for 15 s. The hold doubles each time it trips again, up to 120 s. Its newest provisional stays queued and is sent when the hold ends. A 500 ms reader can keep up with finals alone (about one per 0.75 s), so finals still arrive during the lecture rather than after it.
4. **Hard limit.** If a queue still exceeds 256 items or 2 MiB, it is cleared and the client is resynchronised from shared history, starting after the last final it was sent. The client is not disconnected and loses no finals within the retention window.

This matches the course's guidance on graceful overload: finals are never dropped to make room for provisionals, the newest revision of each segment is kept, and no queue grows without bound.

3.1 The slow-mode trigger (four measured versions)

Getting the trigger right was the main engineering problem, because T3 (slow readers) and T5 (lossy link) pull it in opposite directions. In T5, WebSocket runs over TCP, so a lost packet blocks everything behind it until it is retransmitted (head-of-line blocking). The retransmission wait doubles each time (about 0.2, 0.4, 0.8 s), so several pongs from a *healthy* client can arrive late together. In T3, a trace of a 500 ms reader showed fast pongs for about 8 s while the buffers filled, then about 5.5 s of silence, then two late pongs at once.

Version (commit)	Rule	T5: normal clients wrongly in slow mode	T3: slow readers caught
1	one pong later than 1 s	not measured	yes
2 (bb50641)	two consecutive late pongs	19 / 100	yes
3a (2c0f320)	late pongs spanning 3 s, timed from pong *arrival*	–	**no** (broke T3)
3b (d3142fa)	same, timed from ping *sent* (`now − latency`)	1 (an edge case)	yes
4 final (405da57)	streak measured only up to the latest late pong	1 (a real 4.0 s pong); **0 in all 3 official runs**	all 10 at 6.0 s

Table 1. Each change was driven by a VM measurement (one dev T5 run each, 100 clients).

A pong with latency L proves the client did not read for at least L seconds, so I time a streak from when the ping was sent. Time passing *after* the latest late pong proves nothing. Counting it let a 2.2 s pong plus a fast pong 0.9 s later add up to 3.1 s and trip slow mode. I rejected simply raising a single-pong threshold to about 2.5 s: one pong cannot tell "stopped reading" from an unlucky retransmission, and because retransmission waits double, a harsher link crosses any fixed value.

4. Public experiments

Setup. Ubuntu 26.04.1 LTS VM (UTM with Apple Virtualization on a Mac host; ARM64, 2 CPUs, 4 GB RAM), Python 3.12.16, websockets 16.1.1. Server and clients both ran inside the VM, with `tc/netem` on `lo`. I used the unmodified harness with the spec's commands, 3 independent runs per test, and report the median. All runs were scripted by `tools/run_official.sh`. All 33 runs had **0 duplicate and 0 stale messages**, and **none of the 15**

prescribed runs was cut off (Section 4.2). Raw logs, CSV/JSON and `SUMMARY.txt`: `experiments/official/candidate-1-utm/`. These runs used tag `candidate-1`; the final tag `candidate-2` differs only in the active-segment eviction path and the browser's local retention (Section 3). The deterministic source never has more than about 2 segments open, so that path cannot trigger in T1 to T5, and the harness does not use the browser client.

Test	Final compl.	P50	P95	P99	Fresh (500 ms)	Fresh final	Coverage	Notes
T1 20 cl., 20 s	**100%**	4.42 ms	6.73 ms	7.79 ms	100%	100%	100%	
T2 100 cl., 40 s	**100%**	11.63 ms	19.04 ms	21.16 ms	100%	100%	100%	
T3 10% slow 500 ms	**99.74%**	7.26 ms	2254 ms	7951 ms	94.65%	90.00%	95.18%	normal P95 16.68 ms; slow P95 8927 ms
T4 20% disconnect	**100%**	11.69 ms	18.07 ms	19.97 ms	100%	98.63%	98.42%	recovery P95 5.52 ms; 120 recovered / run
T5 delay_loss	**100%**	267.82 ms	681.91 ms	954.70 ms	88.93%	85.94%	99.88%	0 slow-mode entries
Extra: T5 at 2 Mbit	**100%**	16.75 ms	51.59 ms	57.58 ms	100%	100%	100%	38 segments; 0 slow-mode

Table 2. Official results, prescribed spec commands, median of 3 runs (UTM VM). The 2 Mbit row is an extra test with `bandwidth_2mbit.sh`.

4.1 Interpretation

- **T1/T2 (scale):** P50 rises from about 4 ms to about 12 ms going from 20 to 100 clients, because one event loop sends each update 100 times. P99 stays near 21 ms and everything is fresh.
- **T3 (slow clients):** normal clients are isolated from slow ones: P95 is 16.68 ms and fresh, the same as T2. The overall P95/P99 is dominated by the 10% slow readers, whose lag (about 6 s at P50) is the client library's 16-message buffer (16 × 0.5 s ≈ 8 s), which the server cannot see into. All 90 normal clients got every final in every run. The 0.26% missing is exactly the **last two finals (segments 75 and 76) for each of the 10 slow readers**: they were sent, but were still waiting in the slow readers' buffers when the harness stopped recording about 3 s later. With fewer segments (`--source-segments 60`), so the source ends earlier, T3 is 100%. Coverage below 100% is the slow readers' coalesced provisionals, the intended trade-off.
- **T4 (reconnect):** disconnected clients resume from their cursor and are sent only the missed finals (120 items per run), within 5.5 ms at P95, with no duplicates.
- **T5 (impairment):** the 3% loss appears as *delay*, not missing data, because TCP retransmits. Finals stay at 100%. The ~120 ms netem delay applies in both directions on `lo`, a base round trip of about 240 ms, so P50 is near 268 ms. Retransmission with head-of-line blocking stretches the tail to about 680/950 ms (P95/P99), and freshness falls to about 89% for the same reason. Under 2 Mbit, coalescing keeps P95 near 52 ms with full coverage.

4.2 Hypervisor timing and the capture cutoff

I first ran everything in a Multipass VM (Ubuntu 24.04.5, same CPU and memory). There, the prescribed T1 to T4 runs were all cut off: the harness wrote its results before the source sent `SOURCE_DONE`, so the last segments were never recorded, whatever the server did. I traced it to timer lag in the VM: each `asyncio.sleep(0.2)` in the source took 0.2517 s, and with three waits per segment a segment took about 0.905 s instead of the 0.75 s `run_demo.py` allows for. Even a single client on an idle VM reached only 84.21%, so it was the environment, not server load; the starter was cut off the same way. My TA advised changing the hypervisor, so I moved to UTM with Apple Virtualization and repeated the measurements:

Measurement	Multipass VM	UTM VM (Apple Virtualization)
Mean `asyncio.sleep(0.2)` (20 calls)	0.2517 s	0.2020 s
Source time per segment (3 sleeps)	≈ 0.905 s	≈ 0.756 s
1 client, 20 s, final completeness	84.21%	100% (22/22)
Prescribed runs cut off (T1–T5, 3 each)	12 of 15 (all of T1–T4)	0 of 15
Completeness T1 / T2 / T3 / T4	95.45 / 87.76 / 91.18 / 85.53%	100 / 100 / 99.74 / 100%

T2 P95 / T4 recovery P95	21.47 ms / 4.88 ms	19.04 ms / 5.52 ms

Table 3. Same code (`candidate-1`) and commands on both VMs. Multipass results: `experiments/official/candidate-1/`.

The low completeness on Multipass was therefore entirely a measurement artefact: the server's delivery was unchanged, and latency was similar on both VMs. On Multipass I had also run supplemental tests with `--source-segments` so the source finished in time; all of them reached 100%, which matches what UTM now shows with the unmodified commands.

4.3 Additional stress tests (Multipass VM, one run each, commit b1961d3)

- **Repeated reconnects** (`tools/stress_reconnect.py`, 100 clients × 60 segments). Random mode: 1111 connects, HISTORY P95 2.9 ms. Storm, with all clients reconnecting together every 4 s: 1500 connects, P95 16.1 ms. Both **100/100 complete, 0 duplicate, 0 stale**. The starter on the same script: 0/30 complete.
- **Boundedness:** 600 s with 100 clients and 10% slow readers. Server RSS was 32.3 MB at 1 min and 32.4 MB at 10 min (flat); 100% completeness.
- **Recovery after impairment:** delay_loss applied from 29 to 69 s of a 120 s run. Normal-client P95 was about 21 ms before, 0.5 to 1.0 s during, and **20.9 ms with 100% fresh in the first 10 s window after reset**, with no stale backlog drained afterwards.

5. Before/after comparison

Before is the supplied starter (tag `starter-baseline`); **after** is `candidate-1`. Both use the same prescribed T3/T4 commands on the same Multipass VM, median of 3, so both are equally affected by its capture cutoff (Section 4.2); the comparison is between them, not against Table 2.

Metric	Starter	Final	Change
T3 final completeness	82.89%	91.18%	+8.3 pts
T3 slow-reader P95 caption age	24.8 s	9.5 s	−62%
T3 overall P99 caption age	21.5 s	8.5 s	−61%
T3 normal-client P95	17.3 ms	20.1 ms	+2.8 ms (still 100% fresh)
T4 final completeness	83.95%	85.53%	+1.6 pts
T4 items recovered through HISTORY / run	0	120	recovery works
T4 supplemental (no cutoff)	97.82% (55 seg., 1 run)	100%	no lost finals

Table 4. Starter vs final design.

Single optimisation disabled: I compared coalescing plus the ping window against the bounded FIFO alone, on one T3 run in a dev container. Completeness went from 95.1% to 99.7%, and slow-reader P99 from 24.0 s to 8.1 s; normal P95 stayed under 14 ms in both. The trigger versions in Table 1 are a second ablation: on T5, normal clients wrongly in slow mode went 19 → 1 → 1, and coverage went 95.50% → 99.66% → 99.41%.

Trade-off. Slow readers receive fewer provisional revisions, and none while in slow mode. In exchange, their finals arrive about 2.6× sooner at P95 and nothing they need is lost. The ping window adds a control frame every 2 messages and slightly raises normal latency (about +3 ms at P95). The source's pacing was the same with 1 and 100 clients, so the overhead does not slow the stream.

6. Failures and trade-offs

- **Hidden backlog:** my first coalescing design looked correct in unit tests but did nothing in T3, because buffers outside the server hid the lag. Finding out why coalescing never triggered exposed this, and it led to the ping window.
- **False slow-mode trips:** the two-late-pong rule passed T3 but put 19% of healthy T5 clients in slow mode. My first fix (3a) broke T3, because it reset on fast pongs and timed from arrival. I found each problem by running both T3 and T5 after every change, not only the test I was fixing.
- **Remaining T5 trip:** in one dev run a single healthy client returned a 4.0 s pong. It really did not read for 4 s, so under my definition it was unresponsive, and the cost was 15 s of finals only for that client. I accepted this; the official runs had 0 trips.
- **Rejected shortcut:** during development I was offered the idea of sending live finals to slow clients labelled as HISTORY. The harness processes HISTORY without the simulated slow-reader delay, so it would have

improved the metric by exploiting the measurement and misusing what HISTORY means in the protocol. I rejected it (AI_USAGE.md).

- **Resync versus disconnect:** at the hard limit I resync from history instead of closing the socket. This costs one burst of finals but keeps the client's view complete, without a reconnect cycle.

7. Limitations

1. **Timing depends on the VM.** The harness assumes the source keeps 0.75 s per segment. Under Multipass it did not, and the prescribed runs were cut off (Section 4.2). My official numbers come from the UTM VM, where timing is accurate; I did not modify the harness.
2. **Slow-reader lag floor** of about 6 s at P50 for a 500 ms reader, set by the client library's receive buffer, which the server cannot observe or drop from. At the end of T3 it shows as the last two finals still unread when capture stops.
3. **Detection delay:** a slow reader is caught only after about 6 to 8 s, once the buffers outside the server fill.
4. **Recovery window:** recovery is guaranteed only for absences up to 60 s (70 s retained). An open segment evicted at the active-segment bound loses its later final.
5. **Scale:** a single process on a single core, with one session per instance and no persistence. All results come from VMs on one laptop with uncontrolled host activity.

8. Future work / FYP direction

- Measure reading speed directly with application-level acknowledgements, instead of inferring it from ping timing, and adapt the slow-mode threshold to each client's own round-trip time.
- Use QUIC or WebTransport so that one lost packet does not block unrelated captions (no cross-stream head-of-line blocking), which addresses the T5 tail.
- Fan out across processes or machines (pub/sub), with multiple sessions and persistent lecture history for revision after class.
- Use live ASR as the caption source, plus per-student bookmarks and search over the recalled window: a natural FYP extension.

Appendix A. Reproduction and evidence

- Setup and exact commands: `README.md`. Official runs: `./tools/run_official.sh <label>`; UTM results in `experiments/official/candidate-1-utm/`, Multipass results in `experiments/official/candidate-1/` (environment.txt, unit_tests.txt, SUMMARY.txt, per-test logs, csv_and_json.tar.gz).
- Starter baseline: `experiments/baseline/`. Server-core reconnect check: `experiments/server-core/`.
- Tests: 22 unit/integration tests (`python -m unittest discover tests`) and a headless-browser test with 2 tabs plus a late joiner (`tests/browser_e2e.js`).
- Key commits: starter-baseline 80d08b8; server core 36ed410; browser client 30106e2; coalescing + ping window fbcb49f; slow-mode trigger bb50641 → 2c0f320 → d3142fa → 405da57; history cap 79c9720; tags candidate-1 and candidate-2 (active-segment eviction, 70 s local retention).
- Server log lines: `SLOW_MODE_ENTER client=… hold=… pong=… late_for=…` and `CLIENT_STATS client=… coalesced=… slow_trips=… resyncs=…`.
- Design rationale per stage: `DESIGN_NOTES.md`. AI tool use and corrections: `AI_USAGE.md`.